the external effect, and it can also be used as we're using it here, to prevent an infinite cascade of changing values and re-renders.

You might also need to perform cleanup between renders or when the component unmounts. This can be done with the `useEffect` hook, but the syntax is convoluted. If `useEffect` has a return value, then that return value is itself a function. The returned function is meant to be a cleanup function and is invoked before the next render of the component begins and again when the component unmounts from the React component tree. The cleanup only happens when `useEffect` is invoked, so if you are using the second argument to `useEffect` to only trigger the effect when certain props change, then the cleanup is only performed before the next `useEffect`.

In our case, the return value `() => clearInterval(interval)` is invoked when the component unmounts to prevent the code from hitting the server once a minute until the end of time. Since we have a second argument of `[]`, the `useEffect` hook is never retriggered, and the cleanup only happens when the component unmounts.

Passing Data Through Our React Code

Following our change through, the `VenueBody` component doesn't change much, it just takes in the extra values and passes them along, giving each row its corresponding `RowData`:

chapter_08/04/app/javascript/components/venue_body.tsx

```
import * as React from "react"
import Row from "./row"
import { VenueData } from "./venue"

interface VenueBodyProps {
  concertId: number
  rowCount: number
  seatsPerRow: number
  ticketsToBuyCount: number
  venueData: VenueData
}

const rowItems = ({
```